

TALLER DE PROGRAMACIÓN EN GO

Especificaciones y Rúbricas de Proyectos de Nivel Medio–Avanzado

Cuatro proyectos de sistemas, con controles de uso responsable de IA

Proyectos incluidos

1. Motor de consultas SQL en memoria
2. Mini-Git — sistema de control de versiones
3. Mini-Docker — runtime de contenedores
4. Supervisor de procesos / job scheduler

Contenido

Contenido	2
1. Introducción y objetivos generales	4
Objetivos de aprendizaje transversales	4
Cómo usar este documento	4
2. Uso responsable de inteligencia artificial	5
2.1. Usos permitidos	5
2.2. Usos no permitidos	5
2.3. Controles de verificación de autoría	5
2.4. Consecuencias del incumplimiento	6
3. Entregables comunes y escala de evaluación	7
3.1. Entregables comunes a todos los proyectos	7
3.2. Escala de niveles de desempeño	7
4. Motor de consultas SQL en memoria	8
Contexto y motivación	8
Objetivos de aprendizaje específicos	8
Alcance por hitos	8
Requisitos técnicos y restricciones	9
Entregables específicos	9
Nota anti-autoría por IA	9
Rúbrica de evaluación	9
5. Mini-Git — control de versiones	11
Contexto y motivación	11
Objetivos de aprendizaje específicos	11
Alcance por hitos	11
Requisitos técnicos y restricciones	12
Entregables específicos	12
Nota anti-autoría por IA	12
Rúbrica de evaluación	12
6. Mini-Docker — runtime de contenedores	14
Contexto y motivación	14
Objetivos de aprendizaje específicos	14
Alcance por hitos	14
Requisitos técnicos y restricciones	15
Entregables específicos	15
Nota anti-autoría por IA	15

Rúbrica de evaluación	15
7. Supervisor de procesos / job scheduler	17
Contexto y motivación	17
Objetivos de aprendizaje específicos	17
Alcance por hitos	17
Requisitos técnicos y restricciones	18
Entregables específicos	18
Nota anti-autoría por IA	18
Rúbrica de evaluación	18
Anexo A — Plantilla de bitácora de decisiones	20
Anexo B — Declaración de uso de IA	20

1. Introducción y objetivos generales

Este documento reúne las especificaciones y rúbricas de cuatro proyectos diseñados para un taller de programación en Go de nivel medio-avanzado. Los cuatro comparten un mismo propósito pedagógico: llevar al estudiante más allá del CRUD sobre un framework y confrontarlo con lo que hace distintivo al lenguaje —concurrencia explícita, manejo de errores como valores, composición mediante interfaces, y contacto directo con el sistema operativo y los protocolos.

Cada proyecto está estructurado en hitos incrementales (H1, H2, ...), de modo que el estudiante entregue funcionalidad demostrable en cada etapa y no un único artefacto final. Esta progresión facilita el acompañamiento docente, permite calificar avances parciales y —de forma deliberada— dificulta que el trabajo se resuelva pegando la salida de un modelo de lenguaje sin comprensión.

Objetivos de aprendizaje transversales

- Diseñar y razonar sobre **concurrencia segura** en Go (goroutines, channels, **context**, **sync**), evitando condiciones de carrera y deadlocks.
- Aplicar el **manejo idiomático de errores** y el ciclo de vida de recursos (**defer**, cierre limpio, propagación de contexto).
- Modelar dominios con **interfaces pequeñas y composición**, en lugar de jerarquías rígidas.
- Escribir **pruebas automatizadas** significativas, incluyendo ejecución con el detector de carrera (**go test -race**).
- Documentar **decisiones de diseño** y defenderlas oralmente, demostrando autoría y comprensión.

Cómo usar este documento

La Sección 2 fija la política de uso responsable de IA y los controles de verificación de autoría; es transversal y se aplica por igual a los cuatro proyectos. La Sección 3 describe los entregables comunes y la escala de niveles de desempeño que alimenta todas las rúbricas. Las Secciones 4 a 7 detallan cada proyecto: contexto, objetivos, hitos, requisitos técnicos, entregables específicos y su rúbrica analítica ponderada.

2. Uso responsable de inteligencia artificial

Principio rector

2.1. Usos permitidos

Se permite —y se fomenta— el uso de asistentes de IA como herramienta de consulta y de revisión, siempre que el estudiante conserve el control intelectual del diseño. En particular:

- Consultar **conceptos** generales: qué es un B-tree, cómo funcionan los namespaces de Linux y qué garantiza un WAL.
- Pedir **explicación de los mensajes de error** o de fragmentos de la documentación estándar de Go.
- **Discutir alternativas de diseño** (mutex vs. channel, nested-loop vs. hash join) para luego decidir por cuenta propia y justificar la decisión.
- Generar **casos de prueba de ejemplo** o datos sintéticos, siempre que el estudiante los entienda y pueda adaptarlos.
- Usar el **autocompletado** del editor para código repetitivo (getters, boilerplate), revisando cada sugerencia.
- Solicitar **revisión de estilo** o de la idiomática sobre código ya escrito por el estudiante.

2.2. Usos no permitidos

- Generar el **proyecto completo, un módulo entero o el núcleo lógico** (parser, evaluador, runtime, planificador) y entregarlo sin haberlo escrito y comprendido.
- Pegar el **enunciado del proyecto** en un asistente y entregar su salida, total o parcialmente.
- Delegar en la IA las **decisiones de arquitectura** sin capacidad de explicarlas ni defenderlas.
- Entregar código con **conceptos, librerías o construcciones que el estudiante no sabe explicar** línea por línea.
- Usar IA durante la **defensa oral** o las evaluaciones de comprensión.

2.3. Controles de verificación de autoría

La evaluación incorpora mecanismos que hacen verificable la autoría. Estos controles son obligatorios y su ausencia penaliza según la rúbrica:

Control	En qué consiste y qué evidencia
	Documento breve donde el estudiante registra, por hito, las decisiones de diseño relevantes y su justificación. Debe reflejar el razonamiento propio (ver plantilla en el Anexo A).

Control	En qué consiste y qué evidencia
	El repositorio Git debe mostrar avance progresivo y coherente: commits pequeños, frecuentes y descriptivos. Un único commit masivo o una historia inverosímil son señales de alarma.
	Cada entrega incluye una declaración de qué herramientas de IA se usaron, para qué, y en qué partes (ver plantilla en el Anexo B). Declarar honestamente NO penaliza; ocultarlo sí.
	El estudiante explica su código y realiza un cambio pequeño solicitado por el docente en el momento (p. ej., agregar un operador, cambiar una política de reintento). Es el control de mayor peso.
	Preguntas sobre por qué se eligió una estructura, qué pasa ante un caso borde, o cómo depuraría un fallo específico del propio proyecto.

Sobre la defensa oral

2.4. Consecuencias del incumplimiento

- **Falta de autoría demostrable** en la defensa (no puede explicar/modificar): la parte afectada se califica en cero y el proyecto no supera el nivel «En desarrollo».
- **Uso no declarado de IA** detectado: aplicación del reglamento de honestidad académica de la institución, además de la penalización de rúbrica.
- **Reincidencia**: escalamiento según normativa institucional.

El criterio no es «prohibir la IA», sino garantizar que el estudiante sea el autor y comprenda lo que entrega. Un estudiante que usó IA para aprender y puede defender su proyecto está en regla; uno que la usó para reemplazar su comprensión, no.

3. Entregables comunes y escala de evaluación

3.1. Entregables comunes a todos los proyectos

- **Repositorio Git** con historia incremental de commits y un **README.md** con instrucciones de compilación y ejecución.
- **Código fuente en Go** que compile con `go build ./...` sin advertencias, formateado con `gofmt` y sin observaciones de `go vet`.
- **Suite de pruebas** ejecutable con `go test ./...`; cuando el proyecto tenga concurrencia, debe pasar también con `go test -race ./...`.
- **Bitácora de decisiones** (Anexo A) y **declaración de uso de IA** (Anexo B).
- **Defensa oral** de 15–20 minutos con modificación de código en vivo.

3.2. Escala de niveles de desempeño

Las rúbricas de cada proyecto son analíticas y ponderadas: cada criterio tiene un peso y se califica según la escala de cuatro niveles siguiente. La columna «Indicadores de logro» de cada rúbrica describe el nivel Competente como ancla; los niveles Sobresaliente, En desarrollo e Insuficiente se interpretan a partir de él según esta tabla.

Nivel	Descripción general y banda orientativa
	Cumple el indicador de forma completa y robusta; maneja casos borde, el código es idiomático y limpio, y la defensa demuestra dominio pleno. Aporta algo más de lo pedido con criterio.
	Cumple el indicador esperado con solidez; fallos menores o casos borde no cubiertos. La defensa demuestra comprensión clara. Es el nivel objetivo del curso.
	Cumple parcialmente; funciona en el caso feliz pero falla en robustez, diseño o pruebas. La defensa muestra comprensión incompleta.
	No cumple el indicador, no compila/funciona, o no hay autoría demostrable en la defensa.

Cálculo de la nota

4. Motor de consultas SQL en memoria

Parsing, planificación y ejecución de consultas sobre datos en memoria.

Contexto y motivación

Toda base de datos relacional transforma una cadena SQL en un plan de ejecución que recorre datos y produce filas. Este proyecto pide construir, a escala reducida, ese camino completo: desde el texto de la consulta hasta el resultado. Los datos se cargan desde archivos CSV a tablas en memoria; no hay disco ni transacciones, lo que permite concentrarse en el corazón del problema: lexer, parser, árbol de ejecución y operadores componibles.

El eje conceptual es el modelo de ejecución por iteradores (modelo Volcano): cada operador —escaneo, filtro, proyección, orden, agregación, join— expone una interfaz uniforme y consume filas del operador que tiene debajo. Comprender esta composición es el objetivo central y es lo que hace al proyecto difícil de resolver copiando: el diseño de la interfaz y su encadenamiento exigen decisiones propias.

Objetivos de aprendizaje específicos

- Implementar un **lexer** y un **parser** (descenso recursivo o Pratt) para un subconjunto de SQL, produciendo un AST.
- Diseñar una **interfaz de operador** común (p. ej. `Operator` con `Next()` (`Row`, `error`)) y componer operadores en un árbol de ejecución.
- Aplicar el **modelo iterador (Volcano)**: pull de filas, evaluación perezosa y cierre de recursos.
- Evaluar **expresiones** (comparaciones, AND/OR, aritmética) sobre filas con tipos.
- Implementar **agregación** (`GROUP BY`, `COUNT`, `SUM`, `AVG`) y, en el nivel avanzado, `JOIN`.

Alcance por hitos

Los hitos son acumulativos: cada uno se construye sobre el anterior. Un proyecto sólido alcanza al menos hasta H4; H5 distingue el nivel Sobresaliente.

Hito	Foco	Requisitos funcionales
		Cargar uno o más CSV a tablas en memoria con nombres de columna y tipos (entero, decimal, texto, booleano). Catálogo consultable de tablas y esquemas. Inferencia o declaración explícita de tipos.
		Tokenizar y parsear <code>SELECT <cols> FROM <tabla> [WHERE <cond>]</code> a un AST. Soportar <code>*</code> , lista de columnas, y condiciones con <code>=</code> , <code><></code> , <code><</code> , <code>></code> , <code><=</code> , <code>>=</code> , <code>AND</code> , <code>OR</code> y paréntesis. Errores de sintaxis con mensaje y posición.
		Operadores <code>Scan</code> , <code>Filter</code> (WHERE) y <code>Project</code> (SELECT) implementando la interfaz iterador. Evaluación de expresiones sobre filas. Un REPL o CLI que reciba consultas e imprima resultados tabulados.
		<code>ORDER BY <col> [ASC DESC]</code> , <code>LIMIT n</code> , y agregaciones <code>COUNT/SUM/AVG/MIN/MAX</code> con <code>GROUP BY</code> . Manejo correcto de <code>NULL</code> en comparaciones y agregados.

Hito	Foco	Requisitos funcionales
		INNER JOIN ... ON mediante nested-loop y, además, hash join; comparar ambos. Opcional: DISTINCT , alias de tabla, y un plan textual (EXPLAIN) que muestre el árbol de operadores.

Requisitos técnicos y restricciones

- Lenguaje Go; sin librerías de parsing generadas ni motores SQL externos. Sí se permite la librería estándar (**encoding/csv**, **strings**, **sort**).
- La **interfaz de operador** debe ser genuina: agregar un operador nuevo no debe requerir modificar los existentes (principio abierto/cerrado).
- Manejo de errores explícito: consultas mal formadas o columnas inexistentes devuelven error, no **panic**.
- Pruebas de tabla (**table-driven tests**) que cubran parser y ejecución, incluyendo consultas inválidas.

Entregables específicos

- CLI/REPL ejecutable y un conjunto de CSV de ejemplo.
- Documento de **gramática soportada** (EBNF o descripción precisa del subconjunto SQL).
- Bitácora de decisiones destacando el diseño de la interfaz de operador y el manejo de tipos/NULL.

Nota anti-autoría por IA

--

Rúbrica de evaluación

Criterio	Peso	Indicadores de logro — nivel Competente (referencia de anclaje)
	20%	Lexer y parser cubren el subconjunto especificado; el AST es limpio y separa análisis sintáctico de ejecución; errores de sintaxis informativos con posición.
	25%	Interfaz de operador coherente; operadores componibles y de evaluación perezosa; agregar un operador no obliga a tocar los demás. Recursos se cierran correctamente.
	20%	Alcanza H4 con solidez (WHERE, ORDER BY, LIMIT, GROUP BY y agregados). Manejo correcto de tipos y NULL. H5 (JOIN) para nivel superior.
	15%	Interfaces pequeñas, código idiomático, gofmt/go vet limpios, sin panic en flujo normal, nombres claros.
	10%	Table-driven tests significativos sobre parser y ejecución, incluyendo consultas inválidas y casos borde (tabla vacía, NULLs, grupos).

Criterio	Peso	Indicadores de logro — nivel Competente (referencia de anclaje)
	10%	Explica el flujo de una fila por el árbol, justifica el diseño de la interfaz y agrega un operador/cláusula en vivo. Bitácora coherente.

Escalamiento sugerido

5. Mini-Git — control de versiones

Almacenamiento direccionable por contenido y el grafo de objetos de Git.

Contexto y motivación

Git no es magia: es un almacén de objetos direccionables por contenido sobre un grafo de Merkle. Cada archivo se guarda como un blob identificado por el hash de su contenido; los directorios como trees que referencian blobs y otros trees; y cada commit apunta a un tree y a sus commits padres. Reconstruir este modelo obliga al estudiante a entender por qué Git es inmutable, cómo se deduplican contenidos y qué significa realmente «hacer un commit».

El proyecto reimplementa un subconjunto de Git compatible en concepto (no necesariamente binariamente idéntico) con sus propios comandos. Es un excelente ejercicio de serialización, hashing y estructuras de datos, y resiste bien la resolución por IA: un esqueleto generado no funciona hasta que el estudiante comprende cómo se enlaza el grafo de objetos.

Objetivos de aprendizaje específicos

- Implementar **almacenamiento direccionable por contenido** con hashing (SHA-1 o SHA-256) y compresión (`compress/zlib`).
- Modelar los tres tipos de objeto —**blob**, **tree**, **commit**— y su serialización.
- Construir el **grafo de Merkle**: trees que referencian blobs/trees, commits que referencian tree y padres.
- Implementar un **área de staging (index)** y la operación de commit.
- Recorrer el historial (`log`) siguiendo los punteros de commit padre.

Alcance por hitos

Los hitos construyen el modelo de objetos de abajo hacia arriba. H1–H4 conforman un Git funcional mínimo; H5 añade ramas y navegación.

Hito	Foco	Requisitos funcionales
		<code>init</code> crea la estructura del repositorio (p. ej. <code>.minigit/objects</code>). <code>hash-object</code> calcula el hash del contenido, lo comprime y lo almacena como blob. <code>cat-file</code> recupera y descomprime un objeto por su hash.
		Serializar y almacenar <code>tree</code> que referencian blobs y subtrees con nombre y modo. <code>write-tree</code> toma el estado de un directorio y produce el tree raíz; <code>read-tree/ls-tree</code> lo recorre e imprime.
		Área de staging (index) con <code>add</code> . <code>commit</code> crea un objeto commit que apunta al tree raíz, al commit padre, autor y mensaje. Actualizar la referencia <code>HEAD</code> /rama actual.
		<code>log</code> recorre la cadena de commits desde <code>HEAD</code> mostrando hash, autor, fecha y mensaje. <code>status</code> básico: diferencias entre working dir, index y último commit.

Hito	Foco	Requisitos funcionales
		branch y checkout de ramas; materializar el working directory desde un tree. Opcional: diff entre dos commits y un merge fast-forward.

Requisitos técnicos y restricciones

- Go y librería estándar (**crypto/sha1** o **sha256**, **compress/zlib**, **os**, **io**, **encoding/binary** o formato propio documentado).
- Prohibido invocar el binario **git** o librerías que reimplementen Git; el modelo de objetos debe ser propio.
- El formato de serialización de objetos debe estar **documentado** en el README (aunque no sea idéntico al de Git real).
- Integridad: recuperar un objeto y verificar que su hash coincide con su contenido.

Entregables específicos

- CLI con los comandos implementados y ejemplos de uso en el README.
- Documento del **formato de objetos** (blob/tree/commit) y de la estructura del repositorio.
- Bitácora que explique cómo se enlaza el grafo de Merkle y por qué el hash garantiza inmutabilidad.

Nota anti-autoría por IA

Rúbrica de evaluación

Criterio	Peso	Indicadores de logro — nivel Competente (referencia de anclaje)
	25%	Blob, tree y commit correctamente serializados y direccionados por contenido; el grafo de Merkle se enlaza bien; los hashes son verificables y estables.
	20%	init/hash-object/cat-file/write-tree/add/commit/log funcionan de forma coherente entre sí; el estado del repositorio es consistente.
	20%	Alcanza H4 (commit + log operativos con index). H5 (ramas/checkout) para nivel superior.
	15%	Formato documentado; compresión correcta; recuperación fiel del contenido; manejo de errores ante objetos corruptos o inexistentes.
	10%	Código idiomático, gofmt/go vet limpios, y pruebas que verifican round-trip de objetos y estabilidad de hashes.

Criterio	Peso	Indicadores de logro — nivel Competente (referencia de anclaje)
	10%	Traza el grafo de objetos, explica la inmutabilidad por hash y agrega un comando en vivo. Bitácora coherente e historia de commits creíble.

Escalamiento sugerido

6. Mini-Docker — runtime de contenedores

Aislamiento de procesos con namespaces y cgroups de Linux.

Contexto y motivación

Un contenedor no es una máquina virtual: es un proceso normal de Linux al que el kernel le presenta una vista aislada del sistema mediante namespaces (PID, montajes, red, hostname) y al que se le limitan recursos con cgroups. Este proyecto construye un runtime mínimo que ejecuta un comando dentro de ese aislamiento, revelando qué hace Docker por debajo. Es probablemente el proyecto más cercano al sistema operativo del conjunto.

Go es especialmente adecuado aquí porque expone las banderas de `clone(2)` a través de `syscall.SysProcAttr`. El estudiante enfrentará detalles de bajo nivel —montajes, `pivot_root`, `/proc`, permisos— que no se resuelven pegando código: exigen entender el modelo de procesos de Linux y depurar errores de montaje y capacidades paso a paso.

Objetivos de aprendizaje específicos

- Comprender y aplicar los **namespaces de Linux** (UTS, PID, MNT, y opcionalmente NET/USER) desde Go.
- Ejecutar un proceso hijo aislado con `syscall.SysProcAttr` y `Cloneflags`.
- Montar un **rootfs propio** con `chroot/pivot_root` y un `/proc` independiente.
- Limitar recursos (memoria, CPU) mediante **cgroups**.
- Depurar problemas de bajo nivel de montajes, señales y permisos.

Alcance por hitos

Requiere Linux (una VM o entorno dedicado; no funciona nativamente en macOS/Windows). Los hitos van del aislamiento básico al control de recursos.

Hito	Foco	Requisitos funcionales
		Ejecutar un comando (<code>run <cmd></code>) en un proceso hijo con nuevos namespaces UTS, PID y MNT. Cambiar el hostname dentro del contenedor sin afectar al host, demostrando el aislamiento.
		Montar un rootfs propio (p. ej. un <code>alpine</code> extraído a un directorio) con <code>pivot_root</code> o <code>chroot</code> , y montar un <code>/proc</code> propio para que <code>ps</code> dentro solo vea los procesos del contenedor.
		Crear un cgroup (v2 preferido) para el contenedor y limitar memoria y CPU. Demostrar que un proceso que excede el límite de memoria es controlado por el kernel (OOM).
		CLI con <code>run</code> , paso de variables de entorno, montaje de volúmenes básicos, y limpieza correcta de recursos (namespaces, cgroups, montajes) al terminar el contenedor.
		Configurar un par <code>veth</code> y un namespace de red para dar conectividad al contenedor, o al menos un <code>loopback</code> aislado. Opcional: empaquetar/desempaquetar una imagen como tar.

Requisitos técnicos y restricciones

- Go y el paquete `syscall/golang.org/x/sys/unix`; **entorno Linux obligatorio** (VM, WSL2 con las limitaciones del caso, o servidor).
- Prohibido usar Docker, `runc`, `libcontainer` u orquestadores; el aislamiento debe implementarse con namespaces y cgroups directamente.
- El programa debe **limpiar** montajes y cgroups al finalizar; no debe dejar residuos en el host.
- Documentar los privilegios requeridos (típicamente `CAP_SYS_ADMIN`/root) y las precauciones de seguridad.

Entregables específicos

- CLI ejecutable con instrucciones para preparar el rootfs y correr un contenedor.
- Documento que explique **qué aísla cada namespace** usado y cómo se configuran los cgroups.
- Bitácora con los problemas de bajo nivel encontrados (montajes, `/proc`, permisos) y cómo se depuraron.

Nota anti-autoría por IA

Rúbrica de evaluación

Criterio	Peso	Indicadores de logro — nivel Competente (referencia de anclaje)
	25%	El proceso hijo corre en namespaces propios (al menos UTS, PID, MNT); el aislamiento es verificable (hostname, PIDs, montajes independientes del host).
	20%	<code>pivot_root/chroot</code> correctos y <code>/proc</code> propio montado; dentro del contenedor solo se ven sus procesos y su sistema de archivos.
	20%	Cgroup creado y aplicado; límites de memoria y CPU efectivos y demostrables; el kernel controla al proceso que excede el límite.
	15%	CLI usable; recursos (montajes, cgroups, namespaces) se liberan al terminar; sin residuos ni fugas en el host. Manejo de señales correcto.
	10%	Privilegios requeridos documentados; advertencias de seguridad presentes; explicación clara de cada namespace usado.
	10%	Demuestra aislamiento en vivo, explica <code>pivot_root</code> y cgroups, y realiza un ajuste solicitado. Bitácora de depuración creíble.

Escalamiento sugerido

7. Supervisor de procesos / job scheduler

Ciclo de vida de procesos, señales y políticas de reinicio concurrentes.

Contexto y motivación

Herramientas como `systemd`, `supervisord` o los orquestadores de contenedores comparten un núcleo: lanzan procesos, los vigilan, los reinician según una política y responden a señales para apagarse limpiamente. Este proyecto construye un supervisor que gestiona varios procesos hijos definidos en un archivo de configuración, aplicando políticas de reinicio con backoff exponencial y un apagado ordenado.

Es el proyecto donde la concurrencia idiomática de Go brilla: cada proceso supervisado se modela con goroutines y `context`, y el estado compartido se coordina evitando condiciones de carrera. El reto real —y lo que separa el trabajo genuino del copiado— es el cierre limpio: propagar la cancelación, esperar a los hijos y no dejar procesos zombis ni goroutines colgadas.

Objetivos de aprendizaje específicos

- Gestionar el **ciclo de vida de procesos hijos** con `os/exec` (arranque, monitoreo, terminación).
- Implementar **políticas de reinicio** con backoff exponencial y límites de reintento.
- Manejar **señales** del sistema (`SIGTERM`, `SIGINT`, `SIGHUP`) para apagado ordenado y recarga de configuración.
- Coordinar **concurrencia segura** con goroutines, `context` y `sync`, sin condiciones de carrera ni deadlocks.
- Exponer el **estado** de los procesos supervisados de forma consultable.

Alcance por hitos

Los hitos avanzan del arranque básico al control por API y señales. Debe pasar `go test -race`; la seguridad de concurrencia es central en la evaluación.

Hito	Foco	Requisitos funcionales
		Leer un archivo de configuración (YAML/TOML/JSON) que defina procesos (comando, args, entorno, directorio). Lanzarlos con <code>os/exec</code> y capturar su <code>stdout/stderr</code> a logs.
		Detectar cuándo un proceso termina y reiniciarlo según su política (<code>always</code> , <code>on-failure</code> , <code>never</code>). Cada proceso supervisado corre en su propia goroutine coordinada por <code>context</code> .
		Reinicio con backoff exponencial y tope de reintentos; máquina de estados por proceso (<code>running</code> , <code>backoff</code> , <code>stopped</code> , <code>failed</code>). Evitar tormentas de reinicio.
		<code>SIGTERM/SIGINT</code> : apagado ordenado que propaga la terminación a los hijos (con periodo de gracia antes de <code>SIGKILL</code>) y espera su salida. <code>SIGHUP</code> : recarga de configuración sin reiniciar todo.
		Endpoint HTTP o socket Unix para consultar estado y controlar procesos (<code>start/stop/restart</code>). Opcional: scheduling tipo cron para tareas periódicas.

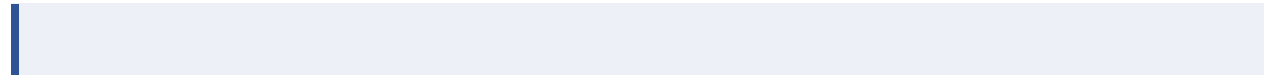
Requisitos técnicos y restricciones

- Go y librería estándar (`os/exec`, `os/signal`, `context`, `sync`, `time`); una librería de parsing de config es aceptable.
- **Obligatorio** pasar `go test -race`. El estado compartido debe protegerse correctamente (mutex o, preferentemente, un modelo por comunicación con channels).
- El apagado debe ser **limpio**: sin procesos zombis, sin goroutines filtradas, con periodo de gracia configurable antes del `SIGKILL`.
- El backoff debe evitar reinicios en bucle inmediato; documentar la política (base, factor, tope).

Entregables específicos

- Binario del supervisor y un archivo de configuración de ejemplo con varios procesos.
- Documento de la **máquina de estados** por proceso y de las políticas de reinicio/backoff.
- Bitácora que explique el modelo de concurrencia elegido (channels vs. mutex) y cómo se garantiza el cierre limpio.

Nota anti-autoría por IA



Rúbrica de evaluación

Criterio	Peso	Indicadores de logro — nivel Competente (referencia de anclaje)
	20%	Arranque, captura de salida, detección de terminación y reinicio según política funcionan de forma fiable con <code>os/exec</code> .
	25%	Pasa <code>go test -race</code> ; el estado compartido está bien coordinado (channels o mutex justificado); sin deadlocks ni goroutines filtradas.
	20%	<code>SIGTERM/SIGINT</code> propagan terminación con periodo de gracia; sin zombis; <code>SIGHUP</code> recarga config. El cierre espera a los hijos.
	15%	Políticas de reinicio correctas; backoff exponencial con tope; máquina de estados clara; evita tormentas de reinicio.
	10%	Código idiomático, <code>gofmt/go vet</code> limpios, y pruebas que ejercitan reinicios, backoff y cancelación por <code>context</code> .
	10%	Explica el modelo de concurrencia y el flujo de cancelación, corre <code>-race</code> en vivo y realiza un cambio solicitado. Bitácora coherente.

Escalamiento sugerido

Anexo A — Plantilla de bitácora de decisiones

El estudiante mantiene esta bitácora a lo largo del proyecto, agregando una entrada por cada decisión de diseño relevante. No busca extensión, sino evidencia de razonamiento propio. Se entrega junto con el código.

Campo	Contenido esperado
	A qué hito corresponde la decisión y cuándo se tomó.
	Qué se decidió (p. ej.: «usar un channel de trabajos en vez de un mutex sobre un slice compartido»).
	Qué otras opciones se evaluaron y por qué se descartaron.
	Razón técnica de la elección, con criterio propio.
	Si se consultó IA para esta decisión, qué se preguntó y qué se hizo con la respuesta (aceptar/adaptar/descartar).

Anexo B — Declaración de uso de IA

Cada entrega incluye esta declaración firmada. Declarar el uso de IA de forma honesta no penaliza; el uso responsable es parte del oficio. Lo que se sanciona es el uso no declarado o la sustitución de la autoría (Sección 2).

Ítem	A completar por el estudiante
	Nombre de los asistentes/herramientas de IA empleados.
	Conceptos, depuración de errores, discusión de diseño, generación de pruebas, revisión de estilo, etc.
	En qué módulos o archivos influyó el uso de IA.
	Qué partes son de autoría íntegra del estudiante (típicamente el núcleo lógico).
	«Declaro que soy autor del diseño y la lógica central de este proyecto, que comprendo todo el código entregado y que puedo explicarlo y modificarlo.» — Firma / nombre.

Cierre